

Table of Contents

Week 9 — Polymorphism

The Third Pillar: One Action, Many Forms

Lesson Overview

Hour Plan

Recap of Week 7 (4 min)

Part 1 — What Is Polymorphism? (11 min)

Open with a story (3 min)

Definition

Connecting to What You Already Built

Three Real-World Polymorphism Scenarios

What Polymorphism Is NOT

Part 2 — Compile-Time Type vs Run-Time Type: The Label and the Box (10 min)

Open with a story (2 min)

The Two Types

Why This Distinction Matters

Part 3 — Polymorphism in Action: One Loop, Many Behaviors (11 min)

Open with a question (2 min)

The Payoff: A Single List, Many Actual Types

The Growth Test

Part 4 — Why Polymorphism Matters: No More Giant if/else (9 min)

Open with a comparison (2 min)

Without Polymorphism (the painful way)

With Polymorphism (the OOP way)

Part 5 — Java as Illustration (8 min)

Reading Polymorphism as a Concept

Polymorphism Appears Where the Parent Type Is Used to Refer to Children

Reading as Plain English

Test Checkpoint

Common Misconceptions

Extension Challenges

● Basic — Spot the Polymorphism

● Intermediate — Trace the Output

Advanced — Design for Growth

Lesson Summary

Homework (Mid-Term Follow-Up / Prep for Abstraction)

Week 9 — Polymorphism

The Third Pillar: One Action, Many Forms

Date: Wednesday, 29 July 2026 | **Time:** 6:30 PM – 7:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Polymorphism — how the same action produces different behavior depending on the actual object type
Date & Time	Wednesday, 29 July 2026 · 6:30 PM – 7:30 PM
Duration	1 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 6 (Inheritance, IS-A), Week 7 (Method Overriding, “actual type decides which version runs”)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can define polymorphism in one sentence, distinguish compile-time type (“label”) from run-time type (“actual object”), explain why a single loop can produce different behavior for different objects, and explain why polymorphism reduces if/else chains

Hour Plan

00:00 – 00:04	Recap of Week 7 + Homework check
00:04 – 00:15	Part 1 – What Is Polymorphism?
00:15 – 00:25	Part 2 – Compile-Time Type vs Run-Time Type: The Label and the Box
00:25 – 00:36	Part 3 – Polymorphism in Action: One Loop, Many Behaviors
00:36 – 00:45	Part 4 – Why Polymorphism Matters: No More Giant if/else

 **Google Meet tip:** Keep `Week9_Summary_Examples.md` open in a second tab — it has a completely different set of real-world stories (payment methods and office printers) ready for every part of this lesson.

Recap of Week 7 (4 min)

Ask students verbally:

1. "What is method overriding, in one sentence?"
→ A child class provides its own version of an action the parent already defined — same name, same inputs.
2. "When we wrote `Person p = sokha;` where `sokha` is a Student, and then called `p.introduce()` — whose version ran? Why?"
→ Student's version ran, because the version that runs is decided by the ACTUAL object type, not the label used to hold it.
3. "From homework — when you introduced 2 students and 2 teachers from a shared `Person` list, did each one produce different output even though you called the exact same action, `introduce()`, on all of them?"
→ Yes — same method call, different output per actual type.
4. Transition question: "We said last week that this idea — 'which version runs is decided while the program is running, not when it was written' — has a name. Any guesses what today's lesson is called?"
→ Polymorphism. That is exactly today's topic — we are naming and formalizing what you already built last week.

Part 1 — What Is Polymorphism? (11 min)

Open with a story (3 min)

"Think about the word 'START' on a button. You will find a START button on a car, a washing machine, a microwave, and a treadmill.

Pressing START always means 'begin your main job' — but what actually happens is completely different each time. The car begins driving. The washing machine begins spinning. The microwave begins heating. The treadmill begins moving its belt.

One instruction — 'START' — many different real behaviors, depending on which machine you pressed it on. That is Polymorphism."

Definition

Polymorphism means “many forms.” In OOP, it is the ability to call the same action on different objects and have each object respond with its own behavior — determined by what the object actually is, not by what it is being called.

Connecting to What You Already Built

YOU ALREADY DID THIS IN WEEK 7 – WE ARE JUST NAMING IT NOW

```
Person p = new Student("Sokha", 20, "CS");
p.introduce();
  → "Hi, I am Sokha (age 20)"
  → "  I study CS"                ← Student's own version ran
```

```
Person p2 = new Teacher("Syuth", 30, "OOP");
p2.introduce();
  → "Hi, I am Syuth (age 30)"
  → "  I teach OOP"              ← Teacher's own version ran
```

SAME CODE: `p.introduce()`
DIFFERENT RESULT: depends on what `p` actually IS
This is Polymorphism.

Three Real-World Polymorphism Scenarios

Work through each one. Ask: “What is the one shared action? What different behaviors does each type produce?”

SCENARIO 1 – HOME APPLIANCES (the START button, from our opening story)

```
Shared action:    start()
Car:              start() → begins driving
WashingMachine:  start() → begins spinning
Microwave:       start() → begins heating
```

Why polymorphism?

Whoever presses START does not need to know the internal mechanism of each appliance – they just trust each one to respond correctly to the same instruction.

SCENARIO 2 – MEDIA FILES (a music/video app)

Shared action: `play()`
Song: `play()` → streams audio through speakers
Video: `play()` → streams audio AND opens a video screen
Podcast: `play()` → streams audio, resumes from last position

Why polymorphism?

The "Play" button in the app is a single button. It calls `play()` on whatever media item is selected, and each media type knows how to play itself correctly.

SCENARIO 3 – VEHICLES (from Week 6 & 7 – fuel cost)

Shared action: `fuelCost(km)`
Car: `fuelCost()` → $\text{km} \times \$0.12 \text{ per km}$
ElectricCar: `fuelCost()` → $\text{km} \times \$0.03 \text{ per km}$

Why polymorphism?

A trip-planning app can call `fuelCost()` on ANY vehicle in a list – car, electric car, or a future vehicle type not even written yet – and always get the correct answer.

What Polymorphism Is NOT

NOT POLYMORPHISM:

Calling two completely different action names

→ `car.drive()` and `washer.spin()` are different names – there is no shared action being reused

Two unrelated classes that happen to share a method name but are not connected by inheritance

→ this looks similar but is not the OOP concept of polymorphism, which depends on an IS-A relationship

POLYMORPHISM:

The SAME action name, called on objects from a shared parent type, produces behavior specific to each object's ACTUAL type.

→ `payment.pay()` – one name, many real behaviors,

because Cash, Card, and QR all IS-A Payment

Part 2 — Compile-Time Type vs Run-Time Type: The Label and the Box (10 min)

Open with a story (2 min)

"Imagine a shipping box labeled 'FRAGILE — GLASSWARE.' That label tells the shipping company how to handle the box — carefully, right-side up.

But what's actually inside the box could be a vase, a mirror, or a picture frame — all different kinds of glassware. The label tells you the general category. Opening the box tells you exactly what's inside — and only what's inside determines how it actually behaves when you use it."

The Two Types

COMPILE-TIME TYPE (the label)

```
Person p = new Student( ... );
```

"Person" is the LABEL –
the type written on the variable

The label limits WHAT YOU CAN ASK
→ you can only call actions that
Person knows about, even though
the real object is a Student

RUN-TIME TYPE (what's in the box)

"Student" is the ACTUAL object –
the type that was really created
with 'new'

The actual object decides
HOW THE ANSWER COMES OUT
→ p.introduce() runs
Student's version, not
Person's generic version

Why This Distinction Matters

"This is the exact rule from Week 7, now with a name: the compile-time type (the label) decides what you're allowed to call. The run-time type (the actual object) decides what actually happens when you call it."

EXAMPLE:

```
Person p = new Student("Sokha", 20, "CS");

p.introduce();    ← allowed, because Person has introduce()
                  RUNS Student's version (run-time type wins)

p.study();        ← NOT allowed, even though the real object
                  IS a Student and DOES have study()
                  Because the LABEL "Person" does not know
                  about study() – the label limits what
                  you can ask for.
```

Part 3 — Polymorphism in Action: One Loop, Many Behaviors (11 min)

Open with a question (2 min)

"If I gave you a mixed box of payment slips — some cash, some card, some QR — and told you 'process every slip the same way, just call pay() on each one' — would you need to open the box first and sort them by type before you could process them?"

Students will realize: No — you can process the whole mixed box the same way, and each slip will handle itself correctly.

The Payoff: A Single List, Many Actual Types

```
Person[] people = {
    new Student("Sokha", 20, "CS"),
    new Teacher("Syuth", 30, "OOP"),
    new Student("Dara", 22, "Engineering"),
    new Person("Chan", 45)
};

for (Person p : people) {
    p.introduce();    // ONE line of code
}
```


OUTPUT – each object handles the SAME call differently:

```
Hi, I am Sokha (age 20)
  I study CS                ← Student's own version ran

Hi, I am Syuth (age 30)
  I teach OOP               ← Teacher's own version ran

Hi, I am Dara (age 22)
  I study Engineering        ← Student's own version ran

Hi, I am Chan (age 45)      ← Person has no override,
                             runs its own default version
```

ONE loop. ONE line of calling code. FOUR different real outcomes.
This is Polymorphism – the concept Week 7 was quietly building
toward, now given its full name and explanation.

The Growth Test

"Here is the real power: if next semester we add a `Robot extends Person` class with its own `introduce()`, do we need to change the loop above AT ALL?"

ANSWER: NO.

```
Add Robot to the array:
  new Robot("UNIT-07")
```

```
The exact same loop:
  for (Person p : people) { p.introduce(); }
```

...automatically produces Robot's own `introduce()` output too –
without touching a single line of the loop.

This is why polymorphism matters: code that works with the
PARENT TYPE automatically works with EVERY current and future
child type, with zero changes.

Part 4 — Why Polymorphism Matters: No More Giant if/else (9 min)

Open with a comparison (2 min)

"Imagine a delivery app that has to calculate delivery cost. Without polymorphism, the app has to ask 'what type of vehicle is this?' every single time before it can act."

Without Polymorphism (the painful way)

```
if (vehicle.getType().equals("Car")) {
    cost = distance * 0.12;
} else if (vehicle.getType().equals("ElectricCar")) {
    cost = distance * 0.03;
} else if (vehicle.getType().equals("Motorbike")) {
    cost = distance * 0.05;
} else if (vehicle.getType().equals("Bicycle")) {
    cost = 0;
}
// Every time a new vehicle type is added, this chain must
// be found and edited, in every place it's used across
// the whole codebase – easy to forget a spot, easy to break.
```

With Polymorphism (the OOP way)

```
cost = vehicle.fuelCost(distance);
// Each vehicle type already knows its own formula.
// Adding a new vehicle type requires ZERO changes here.
```

THE PRINCIPLE:

Without polymorphism: the CALLING CODE must know about every possible type and decide what to do for each one.

With polymorphism: EACH TYPE knows how to handle itself. The calling code just asks, and trusts each object to respond correctly.

This is sometimes summarized as:

"Don't ask what type it is – just tell it what to do,

and let each type figure out how."

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading Polymorphism as a Concept

```
// PARENT - establishes the shared action all children can respond to
public class Person {
    protected String name;
    protected int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void introduce() {
        System.out.println("Hi, I am " + name + " (age " + age + ")");
    }
}

// CHILDREN - each overrides introduce() with its own behavior
public class Student extends Person {
    private String major;

    public Student(String name, int age, String major) {
        super(name, age);
        this.major = major;
    }

    @Override
    public void introduce() {
        super.introduce();
        System.out.println(" I study " + major);
    }
}

public class Teacher extends Person {
    private String subject;

    public Teacher(String name, int age, String subject) {
        super(name, age);
    }
}
```

```

    this.subject = subject;    }                @Override    public void introduce() {
    super.introduce();        System.out.println(" I teach " + subject);    }    }

```

Polymorphism Appears Where the Parent Type Is Used to Refer to Children

```

Person[] people = {
    new Student("Sokha", 20, "CS"),
    new Teacher("Syuth", 30, "OOP"),
    new Person("Chan", 45)
};

for (Person p : people) {    // "p" has the LABEL Person (compile-time type)
    p.introduce();           // the ACTUAL object (run-time type) decides the output
}

```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
Person[] people	A list holding many DIFFERENT actual types, all connected by one parent
Person p (in the loop)	The LABEL – compile-time type, limits what actions you can call
p.introduce()	The ACTUAL object decides which version really runs – run-time type
One loop, many outputs	Polymorphism – "many forms" from one shared instruction

Test Checkpoint

- ☐ Can define polymorphism in one sentence using the START-button or payment-method analogy
- ☐ Can explain the difference between compile-time type ("the label") and run-time type ("the actual object")
- ☐ Can explain why `Person p = new Student( ... )` lets you call `p.introduce()` but not `p.study()`
- ☐ Can trace through a loop over a mixed-type array and predict each object's actual output
- ☐ Can explain, in plain English, why polymorphism removes the need for long if/else type-checking chains

Common Misconceptions

Misconception	Clarification
"Polymorphism and overriding are two separate topics"	They are deeply connected — overriding is the <i>mechanism</i> ; polymorphism is the <i>result</i> . Overriding lets each child define its own version; polymorphism is what happens when you call that action through a shared parent type and the correct version runs automatically.
"The variable's label decides what actually happens"	No — the label only decides what you are <i>allowed to call</i> . The actual object (run-time type) decides what <i>really happens</i> when you call it.
"I need to check the object's type before calling its method"	That is exactly what polymorphism is designed to avoid. If every child correctly overrides the shared action, you never need to ask "what type is this?" — you just call the action and trust it.
"Polymorphism only works with arrays or lists"	No — it works anywhere a parent type is used to refer to a child object, including single variables, method parameters, and return values. Arrays/lists just make the effect very visible.
"Adding a new child type means I have to update all the old code that used the parent type"	The opposite is true — that is the entire benefit. Code written against the parent type keeps working, unchanged, when new child types are added later.

Extension Challenges

Basic — Spot the Polymorphism

For each pair below, decide whether calling the shared action demonstrates polymorphism. Explain why or why not.

- `Animal` has `makeSound()`. `Dog` and `Cat` both override `makeSound()`. You loop over an `Animal[]` array containing both and call `makeSound()` on each.
- A `Calculator` class has `add(int, int)` and `add(double, double)` — you call each directly with matching values.

3. `Shape` has `area()`. `Circle` and `Rectangle` both override `area()`. You call `shape.area()` where `shape` is declared as type `Shape` but actually holds a `Circle`.

🟡 Intermediate — Trace the Output

Given this hierarchy: `Employee` (base pay only) → `Manager` extends `Employee` (adds a bonus, overrides `calculatePay()` to extend the base) → `Intern` extends `Employee` (overrides `calculatePay()` to replace it entirely with a flat stipend).

```
Employee[] staff = {
    new Employee("Chan", 500),
    new Manager("Dara", 800, 200),
    new Intern("Sokha", 150)
};

for (Employee e : staff) {
    System.out.println(e.calculatePay());
}
```

Predict the output for each line, and explain WHY each one behaves differently even though the loop calls the exact same method name on every object.

🔴 Advanced — Design for Growth

Design (in plain English, no code required) a `Notification` parent type with a `send()` action, and three children: `EmailNotification`, `SMSNotification`, `PushNotification` — each overriding `send()` with its own delivery method.

- Write the loop (in plain English) that would send a mixed list of 5 notifications using polymorphism, with only ONE line calling `send()`.
- Now imagine a 4th type, `WhatsAppNotification`, is added six months later. Explain exactly what needs to change in the loop you wrote — and why that answer proves the value of polymorphism.

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. POLYMORPHISM means "many forms" – calling the SAME action on different objects produces behavior specific to each object's ACTUAL type.
2. Two types exist for every variable:

RUN-TIME TYPE (the actual object) – decides what really runs

3. A single loop over a shared parent type can trigger completely different behavior per object, with ZERO type-checking code.
 4. Polymorphism REMOVES the need for long if/else chains that check "what type is this?" – each type handles itself.
 5. New child types can be added LATER without changing any existing code that already works with the parent type. This is the core benefit: code that scales.
 6. Polymorphism is the RESULT. Method overriding (Week 7) is the MECHANISM that makes it possible.
-

Homework (Mid-Term Follow-Up / Prep for Abstraction)

Design the following system in plain English — code is optional (bonus only):

Shape → **Circle**, **Rectangle**, **Triangle** hierarchy with polymorphism:

1. **Shape**: attribute: **name** | action: **area()** (returns 0 — no default formula makes sense for a generic shape)
2. **Circle extends Shape**: adds **radius** | overrides **area()** to REPLACE the parent's version with $\pi \times \text{radius}^2$
3. **Rectangle extends Shape**: adds **width**, **height** | overrides **area()** to REPLACE the parent's version with $\text{width} \times \text{height}$
4. **Triangle extends Shape**: adds **base**, **height** | overrides **area()** to REPLACE the parent's version with $0.5 \times \text{base} \times \text{height}$

Then design a **Shape[]** array holding one of each, and write out — in plain English — what a single loop calling **area()** on every element would print for each shape.

Concept to carry forward: Notice that **Shape.area()** has no sensible default — it only exists so that every child is *forced* to provide its own version. A parent action that exists purely to be overridden, with no real behavior of its own, is the seed of our next topic: **Abstraction** (Weeks 10–11).